

SL MX210 SDK – API Reference Guide

Hallo Welcome ! 😊

This document provides information about everything needed to install MX210 SDK as standalone library structure and establish a connection via Ethernet or with a Serail to communicate and monitor a SL MX210 Optical Analyzer.

MX210 SDK Package Structure:

MX210 SDK package consists of

- Dynamic Link Library *MX210.dll*
- Static Library
 - o Windows : *MX210.lib*
 - o Linux : *libMX210.so*
- Public shared library *mx210.h* to interact with MX210 Analyzer
- Four source codes demonstrating library & function usage of MX210
- CMakeLists.txt for building using cmake build system
 - o Script to build
 - Windows: *build.bat*
 - Linux: *build.sh*

MX210SDK_VersionNr/

Windows	Linux
— bin/ — MX210.dll — include/ — mx210.h — lib/ — MX210.lib — TrialTest/ — src/ — testEthernet_Linux.c — testEthernet_Windows.c — testSerial_Linux.c — testSerial_Windows.c — CMakeLists.txt — build.bat	— include/ — mx210.h — lib/ — libMX210.so — TrialTest/ — src/ — testEthernet_Linux.c — testEthernet_Windows.c — testSerial_Linux.c — testSerial_Windows.c — CMakeLists.txt — build.sh

Platform Requirements:

- SL MX210 Optical Analyzer Firmware Version: >= SL MX210 1.62V
- Target Architecture - x86, x64
- Programming Languages: C, C++17
- Windows:
 - Compiler - Microsoft Visual C++ (MSVC), Visual Studio Build Tools
- Linux:
 - Compiler - Cmake (recommended), gcc, g++
- Tested on: Windows 11 and Ubuntu 24.04

Prerequisites:

MX210 SDK can be built and compiled with several toolchains.

1. Cmake: (Default – both Windows and Linux)
It's a default build system for MX210 SDK, it comes with CMakeLists.txt file building the SDK using cmake. It can be downloaded from <https://cmake.org/download/> based on the platform.
2. GCC : (both Windows and Linux)
 - Windows: GCC can be installed through <https://www.msys2.org/>
 - Linux: Run the following commands

```
sudo apt update
```

```
sudo apt install gcc
```
3. MSVC – (Windows only)
MSVC can be installed as a part of Visual Studio or Visual Studio Build tools by selecting “*Desktop development with C++*” workload. It can be downloaded through the following link <https://visualstudio.microsoft.com/downloads/>.

Quick Setup:

Option 1: (Recommended)

- Clone or Copy MX210 SDK to a destination folder
- MX210 SDK comes with four source codes to interact with MX210 Analyzer
 - o testEthernet_Linux.c – Sample source code demonstrating an Ethernet Communication in Linux
 - o testEthernet_Windows.c – Sample source code demonstrating an Ethernet Communication in Window
 - o testSerial_Linux.c – Sample source code demonstrating a Serial Communication in Linux
 - o testSerial_Windows.c - – Sample source code demonstrating a Serial Communication in Windows

Note: Whenever new .c file is added or an existing .c file is renamed, the corresponding file name must also be updated under add_executable in CMakeLists.txt.

- Build with cmake
 - o Windows: Double click on the batch file *build.bat* inside TrialTest folder
 - o Run “*bash build.sh*” on the command line inside TrialTest folder
- A new folder “build” will be created in TrialTest folder
- On successful build output is at TrialTest/build/Debug/MX210Test
- Run the executables

Option 2:

- Copy MX210.dll to source project
- Copy MX210.lib for linking
- Include mx210.h in the test program
- Compile with any one of the following options
 - o *gcc test_c.c -linclude -L. -lMX210 -o MX210test*
 - o Using CMakeLists.txt (provided)
 - cmake -S . -B build*
 - cmake --build build*
- On successful build executables are created at the intended folder
- Run the executables

Windows	Linux
MX210test.exe	./MX210test

Functions List:

Ethernet Communications -

Function	Function Template	Function Usage	Return	Purpose
Initialize	MX210TcpIP_init (const char* ip)	MX210TCPIP_init (ip)	Device	IP initialization
SetTimeout	MX210TcpIP_setTimeout (MX210TcpIp_t* device, int timeOut)	MX210TCPIP_setTimeout (mx210device, ms)	-1/0	Timeout for connection
Connect	MX210TcpIP_connect (MX210TcpIp_t* device, int port)	MX210TCPIP_connect (mx210device, port)	-1/0	Connect to Ethernet port
Disconnect	MX210TcpIP_disconnect (MX210TcpIp_t* device, int port)	MX210TCPIP_disconnect (mx210device, port)	-1/0	Disconnect from Ethernet port
De-Initialize	MX210TcpIP_deinit (MX210TcpIp_t* device)	MX210TCPIP_deinit (mx210device)	-	De-Register MX210 device
IsAvailable	MX210TcpIP_available (MX210TcpIp_t* device, int port)	MX210TCPIP_available (mx210device, MX210_PORT_8005)	-1/0	Check if socket is available
IsConnected	MX210TcpIP_isConnected (MX210TcpIp_t* device, int port)	MX210TCPIP_isConnected (mx210device, MX210_PORT_8005)	-1/0	Check if socket is connected
ClearBuffer	MX210TcpIP_clearReceiveBuffer (MX210TcpIp_t* device, int port)	MX210TCPIP_clearReceiveBuffer (mx210device, MX210_PORT_8005)	-1/0	Clear Received Buffer

ReadData	MX210TcpIP_readReceivedData (MX210TcpIp_t* device, int port, char* buffer, int bufferSize)	MX210TCPIP_readReceivedData (mx210device, MX210_PORT_8005, buffer, buffersize)	-1/0	Read available data from TCP socket
ReadLine	MX210TcpIP_readLine (char* buffer, int bufferSize, char* line, int lineSize)	MX210TCPIP_readLine (buffer, buffersize, answerbuffer, answerbuffersize)	-1/0	Read whole line of the buffer
SendCommand	MX210TcpIP_sendCommand (MX210TcpIp_t* device, const char* command, int channelNr, int value, char* answerBuffer, int answerBufferSize)	MX210TCPIP_sendCommand (mx210device, "ENA", 1, 1, buffer, buffersize)	-1/0	Send a command and receive response
Send	MX210TcpIP_send (MX210TcpIp_t* device, int port, const void* data, int size)	MX210TcpIP_send (mx210device, MX210_PORT_8000, mx210command, strlen(mx210command))	1/0	Send a command to MX210

Serial:

Function	Function Template	Function Usage	Return	Purpose
Create	MX210SerialPort_Create (void)	MX210SerialPort_Create ()	Handle/NULL	Create a new port handle
Destroy	MX210SerialPort_Destroy (MX210SerialPort_t handle)	MX210SerialPort_Destroy (h)	-	Close handle
GetPorts	MX210SerialPort_GetPorts (void)	MX210SerialPort_GetPorts (buffer, bufferSize)	Ports list array and count	To choose available ports
FreePorts	MX210SerialPort_FreePorts (MX210SerialPortsList* availablelist)	MX210SerialPort_FreePorts (portslist)	-	Free Storage
Open	MX210SerialPort_Open (MX210SerialPort_t h, const char* port, uint32_t baudrate)	MX210SerialPort_Open (h, "COM1", 921600)	1/0	Open a Serial port
Close	MX210SerialPort_Close (MX210SerialPort_t h)	MX210SerialPort_Close (h)	-	Close port
IsOpen	MX210SerialPort_IsOpen (MX210SerialPort_t h)	MX210SerialPort_IsOpen (h)	1/0	Check if port is open
Write	MX210SerialPort_Write (MX210SerialPort_t h, const char* data, size_t len)	MX210SerialPort_Write (h, data, size)	1/0	Send data
Read	MX210SerialPort_Read (MX210SerialPort_t h, char* buf, size_t size, int timeout)	MX210SerialPort_Read (h, data, size, ms)	Read size	Receive Data

API Functions:

Ethernet API Functions:

Function #1: Initialize() - Establish a connection to MX210 device

```
MX210TcpIp_t* tcpIpDevice;  
tcpIpDevice = MX210TcpIP_init("192.168.0.111");  
if (!tcpIpDevice)  
{ fprintf(stderr, "Failed to initialize connection\n"); return 1;}
```

What it does:

- Takes IP address of MX210 device
- Establishes network connection
- Returns MX210 device handle for operations
- Returns NULL if connection fails
- It must be paired with function #De-Initialize

Parameters:

- ip - IP address as string
- Example: "192.168.0.111"

Note:

- Multiple device handles are also possible
- Example:

```
MX210TcpIp_t * mx210device1;  
mx210device1 = MX210TcpIP_init("192.168.0.111");  
MX210TcpIp_t * mx210device2;  
mx210device2 = MX210TcpIP_init("192.168.0.112");  
//Perform operations  
MX210TcpIP_deinit(mx210device1);  
MX210TcpIP_deinit(mx210device2);
```

Function #2: Set Timeout() - Set how long to wait for responses (optional)

```
MX210TcpIP_setTimeout(tcpIpDevice, 1000);
```

What it does:

- Sets timeout for ALL future read operations
- Affects MX210TcpIP_readReceivedData(), MX210TcpIP_sendCommand()
- Returns immediately (no return value)

Parameters:

- MX210 device handle
- timeOut - Wait time in milliseconds (ms)

Function #3: Connect() - Open a communication port on the MX210 Optical Analyzer

```
if(!MX210TcpIP_connect(tcpIpDevice, MX210_PORT_8000))  
{
```

```
printf("PORT 8000 connected!\n");  
MX210TcpIP_deinit(tcpIpDevice);  
}
```

What it does:

- Opens connection to specific port on device
- Returns 1 if successful, 0 if failed
- Must be called before sending commands

Parameters:

- MX210 device handle
- port - Port number

Function #4: IsAvailable() - Check if data is waiting to be read on a port

```
if(MX210TcpIP_available(tcpIpDevice, MX210_PORT_8005))  
{printf("Data is ready");}
```

What it does:

- Checks if data is in receive buffer
- Does NOT block or wait
- Returns immediately, if data available or not

Parameters:

- MX210 device handle
- port - Port number

Function #5: IsConnected() - Checks if port connection is still active

```
If(MX210TcpIP_isConnected(tcpIpDevice, MX210_PORT_8005))  
{ printf("Port is active");}
```

What it does:

- Checks if connection is still valid
- Detects if device disconnected
- Returns, if port is connected or not

Parameters:

- MX210 device handle
- port - Port number

Function #6: SendCommand() - Send command and receive a response from MX210 device

```
char answerBuffer[4096];  
if(!MX210_sendCommand(tcpIpDevice, "VER", NULL, NULL, answerBuffer,  
sizeof(answerBuffer)))  
{printf("%s", answerBuffer);}
```

What it does:

- Sends command to MX210
- Waits for response

- Reads response into buffer
- Returns yes or no

Parameters:

- MX210 device handle
- command name as string
- channelNr of 0-3
- value
- answerBuffer
- size of answerBufferSize

How to Use:

- Specific Optical Channel Configurations
 - It is important to mention command name, channel number MX210_CHx and its value

```
if (!MX210_sendCommand(tcpIpDevice, "ENA", MX210_CH1, 1, answerBuffer,
sizeof(answerBuffer))) printf("%s", answerBuffer);
if (!MX210_sendCommand(tcpIpDevice, "ENA", MX210_CH2, 0, answerBuffer,
sizeof(answerBuffer))) printf("%s", answerBuffer);
```

- All Optical Channels Configurations
 - It is important to mention command name, MX210_ALL indicating all channels and its value

```
if (!MX210_sendCommand(tcpIpDevice, "ANA", MX210_ALL, 4, answerBuffer,
sizeof(answerBuffer))) printf("%s", answerBuffer);
```

- General MX210 Configurations
 - The general configuration that doesn't involve optical channel should have command name followed by NULL and NULL

```
if (!MX210_sendCommand(tcpIpDevice, "STA", NULL, NULL, answerBuffer,
sizeof(answerBuffer))) printf("%s", answerBuffer);
```

Function #7: Send() - Send data (command) without expecting any response to MX210 device

```
char cmd[1024];
printf("> ");
fflush(stdout);
if (!MX210TcpIP_send(tcpIpDevice, MX210_PORT_8000, cmd, strlen(cmd)))
{fprintf(stderr, "\nwrite failed.\n");
break;}
```

What it does:

- Sends anything as command to MX210
- Waits for no response

- Returns if data is sent or not

Parameters:

- MX210 device handle
- Port number
- command as string
- size of command

Function #8: ReadData() - Read raw bytes from a port

```
char answerBuffer[4096], buffer[4096];
if (MX210TcpIP_isConnected(tcpIpDevice, MX210_PORT_8005))
{
    int bytes = MX210TcpIP_readReceivedData(tcpIpDevice, MX210_PORT_8005,
buffer, sizeof(buffer));
    if (bytes > 0)
    {
        buffer[bytes] = '\0'; //NULL Terminate
        printf("Received %d bytes: %s\n", bytes, buffer);
    }
}
```

What it does:

- Reads all available data from buffer
- Blocks up to timeout duration
- Returns number of bytes read

Parameters:

- MX210 device handle
- port - Port number
- a buffer
- size of the buffer

How to Use:

- Null terminate for the read buffer

Function #9: Read Lines() - Extract a line or multiple lines

```
char answerBuffer[4096], buffer[4096];
if (MX210TcpIP_isConnected(tcpIpDevice, MX210_PORT_8005))
{
    MX210TcpIP_readReceivedData(tcpIpDevice, MX210_PORT_8005, buffer,
sizeof(buffer));

    while (MX210TcpIP_readLine(buffer, sizeof(buffer), answerBuffer,
sizeof(answerBuffer)))
        printf("%s", answerBuffer);
}
```

What it does:

- Reads raw data from the read buffer
- Extracts text until newline

- Stops at \n or \r\n
- Stores in line buffer

Parameters:

- buffer - read data from MX210TcpIP_readReceivedData
- size of buffer
- answerbuffer - Store all extracted lines
- size of answerbuffer

Function #10: ClearBuffer() - Empty the receive buffer of a port

```
MX210TcpIP_clearReceiveBuffer(tcpIpDevice, MX210_PORT_8005)
```

What it does:

- Removes all data currently in buffer
- Useful for discarding old/stale data
- Ensures you read fresh data next

When to use:

- Before sending new command to MX210

Parameters:

- MX210 device handle
- port - Port number

Function #11: Disconnect() - Close a port connection of the MX210 Optical Analyzer

```
MX210TcpIP_disconnect(tcpIpDevice, MX210_PORT_8005);
```

What it does:

- Closes connection on specified port
- Flushes any pending data
- Frees port resources

Parameters:

- MX210 device handle
- port – Port number to close

Function #12: De-Initialize() - Disconnect MX210 device from network completely

```
MX210TcpIP_deinit(tcpIpDevice);
```

What it does:

- Closes all open ports and network connection
- Frees all resources
- MX210 device handle becomes invalid
- It must be paired with function #Initialize()

Parameters:

- MX210 device handle

Typical Ethernet Program Flow:

Start

Step1: Initialize IP Address of the MX210 Device

Initialize()

Step2: Optional timeout

SetTimeout()

Step3: Connect to the port

Connect()

Step4: Send Data and receive response

LOOP:

SendCommand() or Send()

IsConnected()

IsAvailable()

ReadData() or with ReadLines()

ClearBuffer()

Step5: Disconnect from the port

Disconnect()

Step6: De-Initialize the MX210 device

De-Initialize()

DONE

Serial API Functions:

Function #1: Create() - Create a Serial port handle

```
MX210SerialPort_t port = MX210SerialPort_Create();  
if(!port){ printf("Error: Unable to create a port\n"); return 1; }
```

What it does:

- Creates a new port handle for you to use
- Returns: Handle or NULL if error

When to use:

- At the very start
- Always pair with function #Destroy() at the end

Function #2: GetPorts() - Scan the available serial ports on the host machine

```
char ports[4096];  
size_t count = MX210SerialPort_GetPorts(ports, sizeof(ports));  
  
if (count == 0){  
    printf("No serial ports found. \n");  
    return 1;  
}
```

```
printf("Found %zu ports:\n", count);
const char* p = ports;
for (size_t i = 0; i < count; i++){
    printf("  %zu. %s\n", i + 1, p);
    p += strlen(p) + 1;
}
```

What it does:

- Scans the host machine for available serial ports
- Returns: Number of ports found
- Fills buffer with port names (Windows: COM3, COM4... | Linux: ttyUSB0, ttyACM0...)

When to use:

- At startup to show user which devices are available
- Before calling function #Open()

Function #3: Free Ports() - Manage dynamic memory for port discovery

```
MX210SerialPort_FreePorts(&ports);
```

What It Does

- Frees memory allocated when discovering available serial ports.

Parameters:

- Ports - Pointer to list from port discovery

When to use:

- After selecting the required port, free remaining allocation
- Don't free twice
- Returns: Nothing

Function #4: Open() - Connect to MX210 Optical Analyzer

```
if(!MX210SerialPort_Open(port, "COM3", 921600))
{
    printf("Error: Could not open COM3.\n");
    MX210SerialPort_Destroy(port); return 1;
}
printf("Connected to COM3\n");
```

What it does:

- Opens connection to a specific port
- Sets communication speed (baud rate)
- Returns: 1 (success) or 0 (failure)

Parameters:

- port - The handle from Create()
- Port name - "COM3" (Windows) or "ttyUSB0" (Linux)
- Baud rate - 921600

When to use:

- To connect to serial port
- Always pair with #Close()

Function #5: Write() - Send a command to MX210 Optical Analyzer

```
const char* cmd = "help\r\n";
if (!MX210SerialPort_Write(port, cmd, strlen(cmd)))
{ printf("Error: Failed to send command\n"); return 1; }
```

What it does:

- Sends bytes to the device
- Waits until all data is sent
- Returns: 1 (success) or 0 (failure)

Parameters:

- port – Serial port to communicate
- cmd – user command to send to MX210
- len – length of the command

How to use:

- Always end commands with “\r\n”. This tells the device message is complete

Function #6: Read() - Reads data from MX210 Optical Analyzer

```
char buffer[256];
size_t bytes = MX210SerialPort_Read(port, buffer, 255, 1000)
buffer[bytes] = '\0';
printf("> %s\n", buffer);
```

What it does:

- Waits for data from MX210
- Fills buffer with received bytes
- Returns: Number of bytes received (0 = timeout)

Parameters:

- buffer - Where to store data
- 255 - Max bytes to read
- 1000 - Timeout in milliseconds

How to use:

- Always add null terminator at the end of the buffer to read

Function #7: IsOpen() - Check if serial connection is still active

```
if (MX210SerialPort_IsOpen(port))
{ printf("Still connected\n"); }
else
{ printf("Connection lost\n"); }
```

What it does:

- Returns: 1 (connected) or 0 (not connected)

Parameters:

- port – Serial port

When to use:

- Check before each Read/Write as a safety check
- Verify device didn't disconnect

Function #8: Close() – End serial connection

```
MX210SerialPort_Close(port);
```

What it does:

- Flushes all pending data
- Closes the connection
- Returns: Nothing

Parameters:

- port – Serial port

Function #9: Destroy() - Clean up and free memory

```
MX210SerialPort_Destroy(port);
```

What it does:

- Releases memory
- Cleans up resources
- Returns: Nothing
- Always pair it with Function #Create()

```
MX210SerialPort_t port = MX210SerialPort_Create(); // Create
// use port
MX210SerialPort_Destroy(port); // Free port
```

Parameters:

- port – Serial port

Typical Serial Program Flow:

START

Step 1: Create handle

port = Create()

Step 2: Get available ports

GetPorts()

Step 3: User selects port

Step 4: Open selected port

Open()

Step 5: Send commands & receive responses

LOOP:

Write()

Read()

```
        Show response
Step 6: Close port
        Close()
Step 7: Free memory
        Destroy()
END
```

Example programs:

This section demonstrates sample source code for Ethernet and Serial Communication. It shows various usage of functions to create an effective communication medium with MX210 Analyzer.

Ethernet Communication Example:

```
#include <stdio.h>
#include "mx210.h"

int main(void)
{
    MX210TcpIp_t* tcpIpDevice;
    int i;
    char answerBuffer[4096], buffer[4096];
    printf("\n\n1. TCP/IP Port 8000 Demo:\n");
    tcpIpDevice = MX210TcpIP_init("192.168.0.111"); //Default IP Address
    if (!MX210TcpIP_connect(tcpIpDevice, MX210_PORT_8000))printf("PORT 8000
connected!\n");

    printf("> Type 'exit' to for Demo 2.\n\n");
    char mx210command[1024];
    printf("> ");
    fflush(stdout);

    while (MX210TcpIP_isConnected(tcpIpDevice, MX210_PORT_8000))
    {
        if (fgets(mx210command, sizeof(mx210command), stdin) == NULL)break;
        size_t len = strlen(mx210command);
        if (len > 0 && mx210command[len - 1] == '\n')mx210command[len - 1]
= '\0';

        if (strcmp(mx210command, "exit") == 0)break;
        if (strlen(mx210command) == 0)
        {
            printf("> ");
            fflush(stdout);
            continue;
        }
    }
}
```

```

        if (MX210TcpIP_isConnected(tcpIpDevice, MX210_PORT_8005))
            strcat(mx210command, "\r\n");

        // if (!MX210TcpIP_sendCommand(tcpIpDevice, "ENA", MX210_EMPTY,
MX210_EMPTY, answerBuffer, sizeof(answerBuffer))) printf("%s",
answerBuffer);
        // Uncomment above if condition to automatically send command
        // or use below if condition to enter a command
        if (!MX210TcpIP_send(tcpIpDevice, MX210_PORT_8000, mx210command,
strlen(mx210command)))
        {
            fprintf(stderr, "\nMX210 write failed.\n");
            break;
        }
        printf("> ");
        fflush(stdout);
    }

    MX210TcpIP_disconnect(tcpIpDevice, MX210_PORT_8000);
    printf("PORT 8000 disconnected\n");

    printf("\n\n2. TCP/IP Port 8005 Demo:\n");
    getch();
    while (1)
    {
        if (MX210TcpIP_isConnected(tcpIpDevice, MX210_PORT_8005))
        {
            MX210TcpIP_readReceivedData(tcpIpDevice, MX210_PORT_8005,
buffer, sizeof(buffer));
            while (MX210TcpIP_readLine(buffer, sizeof(buffer),
answerBuffer, sizeof(answerBuffer)))
                printf("%s", answerBuffer);
        }
        else
        {
            if (!MX210TcpIP_connect(tcpIpDevice, MX210_PORT_8005))
printf("PORT 8005 reconnected\n");
            MX210TcpIP_clearReceiveBuffer(tcpIpDevice, MX210_PORT_8005);
        }
    }
    MX210TcpIP_disconnect(tcpIpDevice, MX210_PORT_8005);
    MX210TcpIP_deinit(tcpIpDevice);
    return 0;
}

```

Serial Communication Example:

```
#include "mx210.h"
#include <stdio.h>

int main()
{
    char serialports[4096];
    MX210SerialPortsList ports = MX210SerialPort_GetPorts();

    MX210SerialPort_t h = MX210SerialPort_Create();

    // Note: This is an example to open a serial port.
    // The User must know the MX210 device serial port number and select
    appropriate one.
    if(ports.count != 0)MX210SerialPort_Open(h, ports.availableports[0],
    921600); // An example open a Serial port

    MX210SerialPort_Write(h, "help\r\n", 6);
    char buffer[4096];
    size_t len = MX210SerialPort_Read(h, buffer, sizeof(buffer), 10);
    buffer[len] = '\0';
    printf("%s\n", buffer);

    MX210SerialPort_Close(h);
    MX210SerialPort_Destroy(h);

    return 0;
}
```

That's it . You are ready to get started with SL MX210 SDK .